第四章 STL 模型切片轮廓计算基础 代码提纲

Triangle.py 文件实现：

```
class Triangle:
    def __init__(self, A, B, C, N = Vector3D(0,0,0)):
    def __str__(self):
    def zMinPnt(self):              # 3 个顶点中的 Z 方向最低点
    def zMaxPnt(self):              # 3 个顶点中的 Z 方向最高点
    def calcNormal(self):           # 计算面片法向量
```

StlModel.py 文件实现：

```
class StlModel:
    def __init__(self):
        self.triangles = []                                  # 三角面片列表
        self.xMin = self.xMax = self.yMin = self.yMax = self.zMin = self.zMax = 0
    def getFacetNumber(self):                                # 获取 STL 模型中的面片数
    def getCoords(self, line):                               # 从文本中提取坐标，被 readStlFil pass
    def readStlFile(self, filepath):                         # 读取 STL 文件，输入文件路径
    def extractFromVtkStlReader(self, vtkStlReader):         # 从 vtkSTLReader 提取面片信息
    def getBounds(self):                                     # 获取模型 6 个方向边界极值
```

GeomAlgo.py 文件继续实现函数：

```
def intersectSegmentPlane(seg, plane)
                                    # 先端和平面求交，返回交点 或 None
def intersectTrianglePlane(triangle, plane)
                                    # 三角面片和平面求交，返回 Segment 或 None
def intersectTriangleZPlane(triangle, z)
                                    # 三角面片和 z 平面求交
def adjustPolygonDirs(polygons):    # 调整多边形方向，输入为轮廓列表
```

Layer.py 文件中实现：

第四章 STL 模型切片轮廓计算基础 代码提纲

```
class Layer:
    def __init__(self, z):
        self.z = z                  # 当前层高度值
        self.segments = []      # 截交线段列表
        self.contours = []          # 多边形轮廓列表（封闭截交线）
```

SliceAlgo.py 文件实现：

```
def intersectStl_brutal(stlModel, layerThk):        # STL 模型截交函数
def linkSegs_brutal(segs):                          # 暴力拼接函数，返回 polys
def writeSlcFile(layers, path):                     # 写 SLC 文件
def readSlcFile(path)                               # 读 SLC 文件
```

SliceModel.py 文件实现：（不做要求）

```
class SliceModel:
    def __init__(self, stlModel, layerThk, sliceAlgo = "brutal"):
        self.stlModel = stlModel    # 层高以及切片方式
        self.layerThk = layerThk
        if sliceAlgo == 'brutal': self.slice_brutal()
        elif sliceAlgo == 'optimal': self.slice_smart()
    def slice_brutal(self):  # 切片函数，brutal
        self.layers = SliceAlgo.intersectStl_brutal(self.stlModel, self.layerThk)
        for layer in self.layers:
            layer.contours = SliceAlgo.linkSegs_brutal(layer.segments)
            SliceAlgo.adjustPolygonDirs(layer.contours)
    def slice_optimal(self): pass           # 切片函数，优化，暂时未实现
    def writeSlcFile(self, path):           # 写 SLC 文件
        SliceAlgo.writeSlcFile(self.layers, path)
    def readSlcFile(self, path):            # 读 SLC 文件
        self.layers = SliceAlgo.readSlcFile(path)
    def drawLayerContours(self, va, start=0, stop=0xFFFF, step=1, clr=(0,0,0),
lineWidth=1):
```